

```
import numpy as np

import matplotlib.pyplot as plt

import tensorflow as tf

from tensorflow import keras

from tensorflow.keras import layers

from sklearn.model_selection import train_test_split


# Set random seeds for reproducibility

np.random.seed(42)

tf.random.set_seed(42)


# 1. Load and preprocess Fashion MNIST dataset

print("Loading Fashion MNIST dataset...")

fashion_mnist = keras.datasets.fashion_mnist

(x_train_full, y_train_full), (x_test, y_test) = fashion_mnist.load_data()


# Normalize pixel values to [0, 1]

x_train_full = x_train_full.astype('float32') / 255.0

x_test = x_test.astype('float32') / 255.0


# Split training data into train and validation sets

x_train, x_val, y_train, y_val = train_test_split(
    x_train_full, y_train_full, test_size=0.2, random_state=42
)


# Reshape data for neural network (flatten the 28x28 images)

x_train_flat = x_train.reshape(x_train.shape[0], -1)

x_val_flat = x_val.reshape(x_val.shape[0], -1)
```

```
x_test_flat = x_test.reshape(x_test.shape[0], -1)
```

```
print(f"Training set: {x_train_flat.shape}")
```

```
print(f"Validation set: {x_val_flat.shape}")
```

```
print(f"Test set: {x_test_flat.shape}")
```

2. Define model creation functions

```
def create_model_without_regularization():
```

```
    """Create a neural network without any regularization"""
```

```
    model = keras.Sequential([
```

```
        layers.Input(shape=(784,)),
```

```
        layers.Dense(256, activation='relu'),
```

```
        layers.Dense(128, activation='relu'),
```

```
        layers.Dense(64, activation='relu'),
```

```
        layers.Dense(10, activation='softmax')
```

```
    ])
```

```
    return model
```

```
def create_model_with_l2_regularization(l2_lambda=0.01):
```

```
    """Create a neural network with L2 regularization"""
```

```
    model = keras.Sequential([
```

```
        layers.Input(shape=(784,)),
```

```
        layers.Dense(256, activation='relu',
```

```
            kernel_regularizer=keras.regularizers.l2(l2_lambda)),
```

```
        layers.Dense(128, activation='relu',
```

```
            kernel_regularizer=keras.regularizers.l2(l2_lambda)),
```

```
        layers.Dense(64, activation='relu',
```

```
            kernel_regularizer=keras.regularizers.l2(l2_lambda)),
```

```
        layers.Dense(10, activation='softmax')
    ])
    return model
```

```
def create_model_with_dropout(dropout_rate=0.5):
    """Create a neural network with Dropout regularization"""
    model = keras.Sequential([
        layers.Input(shape=(784,)),
        layers.Dense(256, activation='relu'),
        layers.Dropout(dropout_rate),
        layers.Dense(128, activation='relu'),
        layers.Dropout(dropout_rate),
        layers.Dense(64, activation='relu'),
        layers.Dropout(dropout_rate),
        layers.Dense(10, activation='softmax')
    ])
    return model
```

```
def create_model_with_both(l2_lambda=0.01, dropout_rate=0.5):
    """Create a neural network with both L2 and Dropout regularization"""
    model = keras.Sequential([
        layers.Input(shape=(784,)),
        layers.Dense(256, activation='relu',
                     kernel_regularizer=keras.regularizers.l2(l2_lambda)),
        layers.Dropout(dropout_rate),
        layers.Dense(128, activation='relu',
                     kernel_regularizer=keras.regularizers.l2(l2_lambda)),
        layers.Dropout(dropout_rate),
```

```

layers.Dense(64, activation='relu',
              kernel_regularizer=keras.regularizers.l2(l2_lambda)),
layers.Dropout(dropout_rate),
layers.Dense(10, activation='softmax')
])
return model

```

3. Training function

```
def train_model(model, model_name, epochs=30, batch_size=32):
```

```
    """Train a model and return training history"""
```

```
    model.compile(
        optimizer='adam',
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )
```

```
    history = model.fit(
        x_train_flat, y_train,
        validation_data=(x_val_flat, y_val),
        epochs=epochs,
        batch_size=batch_size,
        verbose=0 # Set to 1 to see training progress
    )
```

```
    return history
```

4. Train all models

```
print("\nTraining models...")
```

```
print("=" * 50)
```

```
# Model without regularization
```

```
print("Training model without regularization...")
```

```
model_no_reg = create_model_without_regularization()
```

```
history_no_reg = train_model(model_no_reg, "No Regularization")
```

```
# Model with L2 regularization
```

```
print("Training model with L2 regularization...")
```

```
model_l2 = create_model_with_l2_regularization(l2_lambda=0.01)
```

```
history_l2 = train_model(model_l2, "L2 Regularization")
```

```
# Model with Dropout
```

```
print("Training model with Dropout...")
```

```
model_dropout = create_model_with_dropout(dropout_rate=0.5)
```

```
history_dropout = train_model(model_dropout, "Dropout")
```

```
# Model with both
```

```
print("Training model with both L2 and Dropout...")
```

```
model_both = create_model_with_both(l2_lambda=0.01, dropout_rate=0.5)
```

```
history_both = train_model(model_both, "Both Regularizations")
```

```
# 5. Visualize results
```

```
fig, axes = plt.subplots(2, 2, figsize=(15, 10))
```

```
# Plot training loss
```

```
axes[0, 0].plot(history_no_reg.history['loss'], label='Train', linewidth=2)
```

```
axes[0, 0].plot(history_no_reg.history['val_loss'], label='Validation', linewidth=2,  
linestyle='--')
```

```
axes[0, 0].set_title('No Regularization', fontsize=14, fontweight='bold')
```

```
axes[0, 0].set_xlabel('Epoch')
```

```
axes[0, 0].set_ylabel('Loss')
```

```
axes[0, 0].legend()
```

```
axes[0, 0].grid(True, alpha=0.3)
```

```
# Plot L2 regularization loss
```

```
axes[0, 1].plot(history_l2.history['loss'], label='Train', linewidth=2)
```

```
axes[0, 1].plot(history_l2.history['val_loss'], label='Validation', linewidth=2, linestyle='--  
' )
```

```
axes[0, 1].set_title('L2 Regularization ( $\lambda=0.01$ )', fontsize=14, fontweight='bold')
```

```
axes[0, 1].set_xlabel('Epoch')
```

```
axes[0, 1].set_ylabel('Loss')
```

```
axes[0, 1].legend()
```

```
axes[0, 1].grid(True, alpha=0.3)
```

```
# Plot Dropout loss
```

```
axes[1, 0].plot(history_dropout.history['loss'], label='Train', linewidth=2)
```

```
axes[1, 0].plot(history_dropout.history['val_loss'], label='Validation', linewidth=2,  
linestyle='--')
```

```
axes[1, 0].set_title('Dropout (rate=0.5)', fontsize=14, fontweight='bold')
```

```
axes[1, 0].set_xlabel('Epoch')
```

```
axes[1, 0].set_ylabel('Loss')
```

```
axes[1, 0].legend()
```

```
axes[1, 0].grid(True, alpha=0.3)
```

```
# Plot both regularizations loss
```

```

axes[1, 1].plot(history_both.history['loss'], label='Train', linewidth=2)
axes[1, 1].plot(history_both.history['val_loss'], label='Validation', linewidth=2,
linestyle='--')
axes[1, 1].set_title('L2 + Dropout', fontsize=14, fontweight='bold')
axes[1, 1].set_xlabel('Epoch')
axes[1, 1].set_ylabel('Loss')
axes[1, 1].legend()
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```

6. Compare final results

```

print("\n" + "=" * 50)
print("FINAL RESULTS COMPARISON")
print("=" * 50)

```

```

def evaluate_model(model, model_name):
    """Evaluate model on validation and test sets"""
    val_loss, val_acc = model.evaluate(x_val_flat, y_val, verbose=0)
    test_loss, test_acc = model.evaluate(x_test_flat, y_test, verbose=0)
    return val_loss, val_acc, test_loss, test_acc

```

```

models = [
    ("No Regularization", model_no_reg),
    ("L2 Regularization", model_l2),
    ("Dropout", model_dropout),
    ("Both (L2 + Dropout)", model_both)
]

```

```
]
```

```
print(f"{'Model':<25}{'Val Loss':<12}{'Val Acc':<12}{'Test Loss':<12}{'Test Acc':<12}")
```

```
print("-" * 75)
```

```
for name, model in models:
```

```
    val_loss, val_acc, test_loss, test_acc = evaluate_model(model, name)
```

```
    print(f"{'name':<25}{'val_loss':<12.4f}{'val_acc':<12.4f}{'test_loss':<12.4f}{'test_acc':<12.4f}")
```

```
# 7. Calculate overfitting metrics
```

```
print("\n" + "=" * 50)
```

```
print("OVERFITTING ANALYSIS")
```

```
print("=" * 50)
```

```
print("(Training Loss - Validation Loss difference)")
```

```
print("-" * 50)
```

```
for name, model in models:
```

```
    history = None
```

```
    if name == "No Regularization":
```

```
        history = history_no_reg
```

```
    elif name == "L2 Regularization":
```

```
        history = history_l2
```

```
    elif name == "Dropout":
```

```
        history = history_dropout
```

```
    else:
```

```
        history = history_both
```



```
train_loss = history.history['loss'][-1]
val_loss = history.history['val_loss'][-1]
diff = train_loss - val_loss

print(f"{name:<25} Train: {train_loss:.4f}, Val: {val_loss:.4f}, Diff: {diff:.4f}")
```

8. Additional visualization - Accuracy comparison

```
plt.figure(figsize=(12, 5))
```

Plot validation accuracy

```
plt.subplot(1, 2, 1)
plt.plot(history_no_reg.history['val_accuracy'], label='No Reg', linewidth=2)
plt.plot(history_l2.history['val_accuracy'], label='L2', linewidth=2)
plt.plot(history_dropout.history['val_accuracy'], label='Dropout', linewidth=2)
plt.plot(history_both.history['val_accuracy'], label='Both', linewidth=2)
plt.title('Validation Accuracy Comparison', fontsize=14, fontweight='bold')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True, alpha=0.3)
```

Plot test accuracy comparison

```
plt.subplot(1, 2, 2)
model_names = [name for name, _ in models]
test_accuracies = []
for _, model in models:
    _, _, test_acc = evaluate_model(model, "")
    test_accuracies.append(test_acc)
```

```

bars = plt.bar(model_names, test accuracies, color=['red', 'blue', 'green', 'purple'])
plt.title('Test Accuracy Comparison', fontsize=14, fontweight='bold')
plt.ylabel('Accuracy')
plt.ylim(0.85, 0.92)
plt.xticks(rotation=45)
for bar, acc in zip(bars, test accuracies):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.002,
             f'{acc:.4f}', ha='center', va='bottom')

plt.tight_layout()
plt.show()

```

```
print("\n" + "=" * 50)
```

```
print("KEY OBSERVATIONS:")
```

```
print("=" * 50)
```

```
print("""
```

1. No Regularization: Shows large gap between training and validation loss → OVERFITTING
2. L2 Regularization: Reduces overfitting by penalizing large weights
3. Dropout: Forces network to learn redundant representations by randomly dropping neurons
4. Combined approach: Often provides best generalization by addressing overfitting from multiple angles

Regularization techniques help models generalize better by:

- Preventing overfitting to training data
- Reducing model complexity
- Improving performance on unseen data

""")